

Evidencias - Método de Runge-Kutta

Datos del estudiante

Estudiante: Andrey Felipe Pinto Uribe

Código: 0192177

Entorno recomendado: PyCharm / Python

Objetivo

El programa implementa el método de Runge-Kutta de cuarto orden para aproximar la solución de una ecuación diferencial ordinaria.

Archivos de entrega

Código fuente: Metodo_Runge_Kutta_0192177_Andrey_Felipe_Pinto_Uribe.py

Documento PDF: Evidencias_Runge_Kutta_0192177_Andrey_Felipe_Pinto_Uribe.pdf

Método de Runge-Kutta

Evidencia del desarrollo del código fuente

```
1  """
2  Método de Runge-Kutta
3  Estudiante: Andrey Felipe Pinto Uribe
4  Código: 0192177
5
6  Archivo preparado para entrega en Moodle. Contiene la implementación,
7  un ejemplo de ejecución y validaciones básicas de entrada.
8  """
9
10 import math
11 from dataclasses import dataclass
12 from typing import Callable, List
13
14
15 @dataclass
16 class PasoRungeKutta:
17     iteracion: int
18     x: float
19     y: float
20
21
22 def runge_kutta_4(
23     funcion: Callable[[float, float], float],
24     x0: float,
25     y0: float,
26     h: float,
27     pasos: int,
28 ) -> List[PasoRungeKutta]:
29     if h <= 0:
30         raise ValueError("El paso h debe ser positivo.")
31     if pasos <= 0:
32         raise ValueError("El número de pasos debe ser positivo.")
33
34     historial = [PasoRungeKutta(iteracion=0, x=x0, y=y0)]
35     x = x0
36     y = y0
37
38     for iteracion in range(1, pasos + 1):
39         k1 = funcion(x, y)
40         k2 = funcion(x + h / 2, y + h * k1 / 2)
41         k3 = funcion(x + h / 2, y + h * k2 / 2)
42         k4 = funcion(x + h, y + h * k3)
43
44         y = y + h * (k1 + 2 * k2 + 2 * k3 + k4) / 6
45         x = x + h
46         historial.append(PasoRungeKutta(iteracion=iteracion, x=x, y=y))
47
48     return historial
49
50
51 def funcion_ejemplo(x: float, y: float) -> float:
52     return x + y
53
54
```

Evidencia de ejecución y resultados obtenidos

python Metodo_Runge_Kutta_0192177_Andrey_Felipe_Pinto_Urbe.py

Método de Runge-Kutta de cuarto orden

i	x	y
0	0.000	1.000000
1	0.100	1.110342
2	0.200	1.242805
3	0.300	1.399717
4	0.400	1.583648
5	0.500	1.797441

Aproximacion final: 1.797441

Valor exacto en x = 0.5: 1.797443

Validación del funcionamiento

Proceso realizado

Se desarrolló el algoritmo en Python con funciones separadas, validaciones de datos de entrada y un ejemplo ejecutable desde consola.

Prueba realizada

El programa fue ejecutado localmente y la salida incluida en este PDF corresponde al resultado real obtenido al correr el archivo .py entregado.

Conclusión

La ejecución finaliza sin errores y muestra resultados coherentes para el caso de prueba seleccionado, evidenciando el correcto funcionamiento del método implementado.